

GESTOCK

ARQUITECTURA · CLIENTE <-> MID <-> API · POSTGRESQL

34 DIAGRAMAS EN APAIZADO: 1 POR CONTROLADOR + TRANSVERSALES DE MID/ERRORES/BD

CLIENTE - Angular 18 · interceptor + guards (Bearer, 401, permisos)

MID - rateLimit -> validate (Zod) -> authenticate (JWT+sesion) -> authorize

API - Express 5 · 18 controladores · ApiError + errorHandler

DATOS - PostgreSQL Gestock_db · 18 tablas · multitenant empresa_id

ERRORES - 400/401/403/404/409/429/500 con code (ApiError | PG mapping)

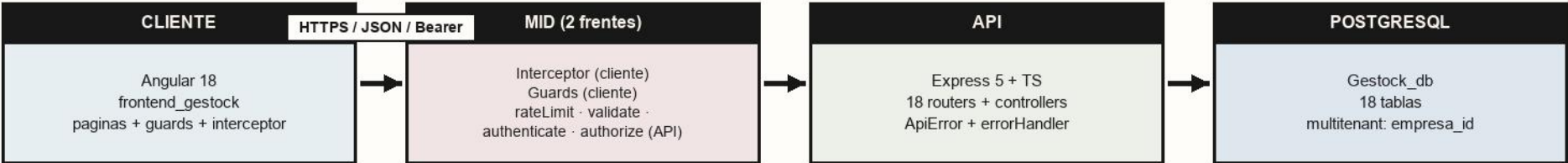
El MID integra AMBOS lados: valida la peticion que ENTRA; la respuesta SALE directo del API al cliente.

Diagramas generados desde el codigo fuente real del proyecto

ARQUITECTURA GENERAL GESTOCK

GESTOCK · ARQUITECTURA Y FLUJOS

CLIENTE (Angular) -> MID (front+back) -> API (Express) -> PostgreSQL



DETALLE DEL MID (no ficticio)

EN EL CLIENTE (Angular)

- api.service.ts: devuelve res.data
- api.interceptor.ts: agrega Authorization: Bearer <jwt>
- maneja 401 (-> login) y demas errores (toast)
- guards: permissionGuard / roleGuard / homeGuard

EN EL API (Express)

- apiRateLimiter (500 / 15 min)
- validate (Zod schemas)
- authenticate (JWT + sesion activa)
- authorize (permisos del rol)
- errorHandler normaliza ApiError / PG

18 CONTROLADORES REALES (routes/ de backend/src)

auth	usuario	rol	permiso	empresa	categoria
bodega	producto	movimiento	recepcion	auditoria	incidencia
mantenimiento	sesion	notificacion	configuracion	dashboard	reporte

LA CAPA MID EN GESTOCK (DETALLE) GESTOCK · ARQUITECTURA Y FLUJOS

Frontera de seguridad entre el navegador y los controladores

FLUJO DE UNA PETICION AUTENTICADA



ARCHIVOS REALES DE LA CAPA MID

Archivo	Responsabilidad
frontend_gestock/src/app/interceptors/api.interceptor.ts	Agrega Authorization: Bearer <jwt>; 401 != login -> limpiar + toast 'Tu sesion ha expirado' + /auth/login; otros -> toast
frontend_gestock/src/app/guards/permission-guard.ts	refrescarSesion -> estaAutenticado -> data.permission/anyPermission -> toast 'Acceso denegado'
frontend_gestock/src/app/guards/home-guard.ts · role-guard.ts	Redirige por rol a la ruta inicial / restringe acceso
backend/src/middleware/validate.ts	parsea con schema Zod de models/schemas.ts: 400 BAD_JSON / 400 BAD_REQUEST
backend/src/middleware/authenticate.ts	verifica token JWT + sesion activa en BD: 401 UNAUTHORIZED
backend/src/middleware/authorize.ts	permiso del rol (rol_permiso): 403 FORBIDDEN
backend/src/middleware/rateLimit.ts	apiRateLimiter(500), loginEmailLimiter(5), loginIpLimiter(60), recuperacionLimiter

ERRORES DEL MID: CADA FALLA ES UNA RAMA INDEPENDIENTE



AUTENTICACION Y SESION GESTOCK · ARQUITECTURA Y FLUJOS

Login con rateLimit -> JWT 8h -> sesiones activas -> logout

POST /auth/login (auth.routes.ts)

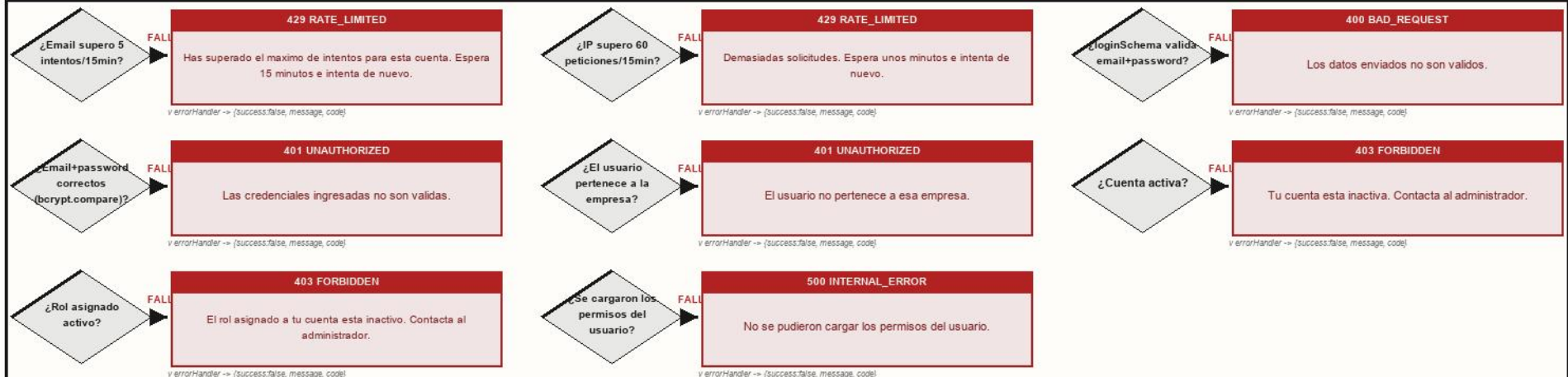


RUTAS PROTEGIDAS CON MIDDLEWARE

Ruta	Middleware / comportamiento
GET /auth/perfil	authenticateToken
POST /auth/logout	authenticateToken -> cierra sesion (Activa->Cerrada)
PUT /auth/contrasena	authenticateToken + validateBody(cambiarContrasenaSchema)
POST /auth/recuperar	recuperacionLimiter + validateBody(solicitarRecuperacionSchema) -> email con codigo
POST /auth/recuperar/validar	validateBody(validarCodigoRecuperacionSchema)
POST /auth/recuperar/restablecer	validateBody(restablecerPasswordSchema)

FRONTEND (MID): localStorage 'gestock_token' + 'gestock_usuario_sesion'; api.interceptor agrega Authorization: Bearer <jwt> y en 401 (ruta != login) redirige a /auth/login con toast.

RAMAS DE ERROR DE LOGIN (cada falla una rama -> errorHandler)



RBAC: ROLES Y PERMISOS

GESTOCK · ARQUITECTURA Y FLUJOS

Roles -> rol_permiso -> permisos -> authorizePermission(...)



PERMISOS REALES POR OBSERVABLE (05_insert_permissions.sql)

Modulo	Permisos (patron modulo.accion)
Dashboard	dashboard.view
Empresas	empresas.view · create · edit · delete
Usuarios	usuarios.view · create · edit · desactivar · password · delete
Roles / Permisos	roles.view · create · edit · delete · asignar_permisos · permisos.view
Categorias / Bodegas	categorias.view · create · edit · delete · bodegas.view · create · edit · delete
Productos / Inventario	productos.view · create · edit · delete · inventario.view
Movimientos / Recepcion	movimientos.view · recepcion.view · create · edit · delete · historial.view
Auditoria / Incidencias / Mantenimiento	auditoria.view · incidencias.view · create · edit · delete · mantenimiento.view · create · edit · delete
Sesiones / Notificaciones / Config / Reportes	sesiones.view · delete · notificaciones.view · configuracion.view · edit · reportes.view

COMO SE APLICA EN BACKEND

authorizePermission('modulo.accion') · authorizeAnyPermission('a','b')

Cada ruta protegida llama authorizePermission con el permiso exacto. El frontend usa los permisos del login para mostrar/ocultar y los guards refuerzan con data.permission o data.anyPermission.

PATRON CRUD TIPO EN GESTOCK

GESTOCK · ARQUITECTURA Y FLUJOS

Un controlador = un modulo · Create/Read/Update/Delete con auditoria

CICLO DE VIDA DE UN REGISTRO (ej. Producto)

LISTAR GET /:ruta	CREAR POST /:ruta	DETALLE GET /:ruta/:id	EDITAR PUT /:ruta/:id	ESTADO PATCH /:ruta/:id	ELIMINAR DELETE /:ruta/:id
<pre>authorize('x.view') listar() -> repo.listar</pre>	<pre>authorize('x.create') validateBody -> crear()</pre>	<pre>authorize('x.view') validateParams -> detalle()</pre>	<pre>authorize('x.edit') validateBody -> actualizar()</pre>	<pre>authorize('x.edit') cambiarEstado()</pre>	<pre>authorize('x.delete') eliminar()</pre>

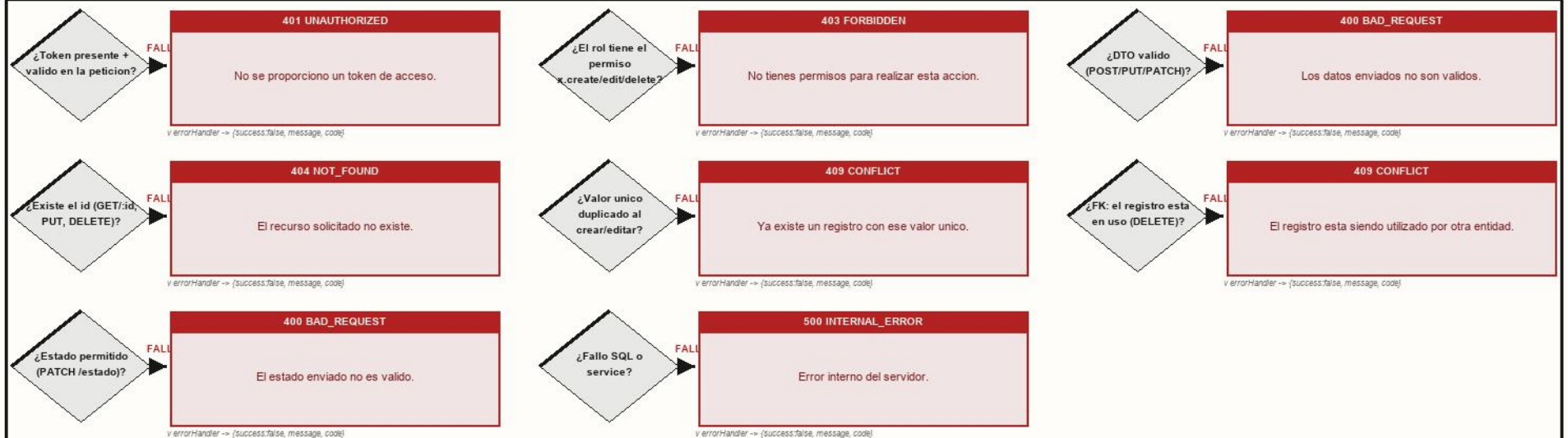
CODIGO TIPO EN UN CONTROLLER (usuario.controller.ts)

```
async listar(req, res, next) { try { const lista = await usuarioService.listar(req.usuarioId); ok(res, lista); } catch (e) { next(e); } }
async crear(req, res, next) { try { const nuevo = await usuarioService.crear(req.body); created(res, nuevo); } catch (e) { next(e); } }
```

AUDITORIA Y RESPUESTAS

Cada operacion relevante inserta en auditorias (usuario_id, accion, entidad, id_registro). Respuestas: ok() 200 · created() 201 · noContent() 200. Errores: next(e) -> errorHandler.

RAMAS DE ERROR DEL PATRON (cada falla su rama -> errorHandler)



MANEJO DE ERRORES (BACKEND) GESTOCK · ARQUITECTURA Y FLUJOS

ApiError + errorHandler + PostgreSQL codes · cada error es su propia rama

CADA CODIGO DE ERROR ES UNA RAMA INDEPENDIENTE -> errorHandler



BASE DE DATOS GESTOCK (PostgreSQL) GESTOCK · ARQUITECTURA Y FLUJOS

18 tablas + relaciones (multitenant: empresa_id en tablas de negocio)

TABLAS REALES DE GESTOCK

NUCLEO / IDENTIDAD	CATALOGOS	INVENTARIO	OPERACIONES	SEGURIDAD / CONFIG
empresas	categorias	productos	recepciones	auditorias
roles	bodegas	movimientos_inventario	recepcion_detalle	notificaciones
permisos			incidencias	configuracion_sistema
rol_permiso			mantenimientos	
usuarios				
sesiones				
restablecimientos_contraseña				

RELACIONES PRINCIPALES

Relacion (FK)
usuarios.rol_id -> roles.id · usuarios.empresa_id -> empresas.id
rol_permiso.rol_id -> roles.id · rol_permiso.permiso_id -> permisos.id
productos.categoria_id -> categorias.id · productos.bodega_id -> bodegas.id
movimientos_inventario.producto_id -> productos.id · bodega_id -> bodegas.id
recepcion_detalle.recepcion_id -> recepciones.id · producto_id -> productos.id
incidencias/mantenimientos .usuario_id -> usuarios.id · auditorias.usuario_id -> usuarios.id

MATRIZ DE TRAZABILIDAD: CONTROLADOR -> MODULO -> PERMISOS

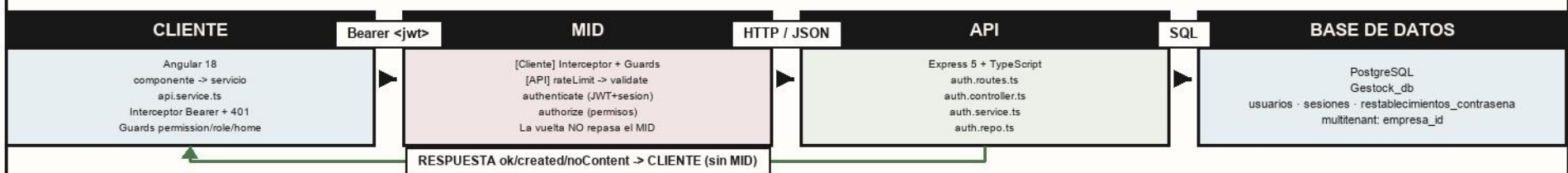
Resumen de los 18 controladores reales

Controlador	Modulo	Permisos que protegen sus rutas (reales)
auth	auth	
usuario	usuario	usuarios.create · usuarios.delete · usuarios.desactivar · usuarios.edit · usuarios.password · usuarios.view
rol	rol	roles.asignar_permisos · roles.create · roles.delete · roles.edit · roles.view
permiso	permiso	permisos.view
empresa	empresa	empresas.create · empresas.delete · empresas.edit · empresas.view
categoria	categoria	categorias.create · categorias.delete · categorias.edit · categorias.view
bodega	bodega	bodegas.create · bodegas.delete · bodegas.edit · bodegas.view
producto	producto	inventario.view · productos.create · productos.delete · productos.edit · productos.view
movimiento	movimiento	movimientos.view · recepcion.create · recepcion.delete
recepcion	recepcion	recepcion.create · recepcion.delete · recepcion.edit · recepcion.view
auditoria	auditoria	auditoria.view
incidencia	incidencia	incidencias.create · incidencias.delete · incidencias.edit · incidencias.view
mantenimiento	mantenimiento	mantenimiento.create · mantenimiento.delete · mantenimiento.edit · mantenimiento.view
sesion	sesion	sesiones.delete · sesiones.view
notificacion	notificacion	notificaciones.view
configuracion	configuracion	configuracion.edit · configuracion.view
dashboard	dashboard	dashboard.view
reporte	reporte	reportes.view

CLIENTE -> MID -> API -> BASE DE DATOS · AUTH

GESTOCK · ARQUITECTURA Y FLUJOS

Autenticacion / Auth · Login, perfil, logout, cambio de contraseña y recuperacion de credenciales (solicitar, validar, restablecer).



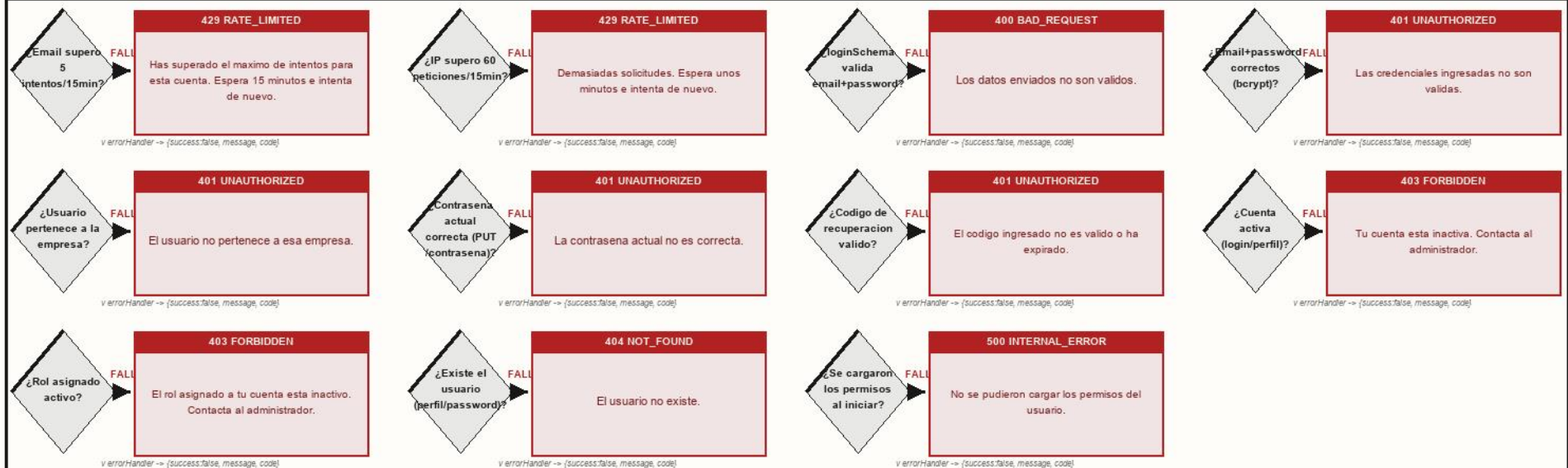
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
POST	/auth/login	validateBody(loginSchema) + loginEmailLimiter + loginIpLimiter
GET	/auth/perfil	authenticateToken
POST	/auth/logout	authenticateToken
PUT	/auth/contrasena	authenticateToken + validateBody(cambiarContrasenaSchema)
POST	/auth/recuperar	recuperacionLimiter + validateBody(solicitarRecuperacionSchema)
POST	/auth/recuperar/validar	validateBody(validarCodigoRecuperacionSchema)
POST	/auth/recuperar/restablecer	validateBody(restablecerPasswordSchema)

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



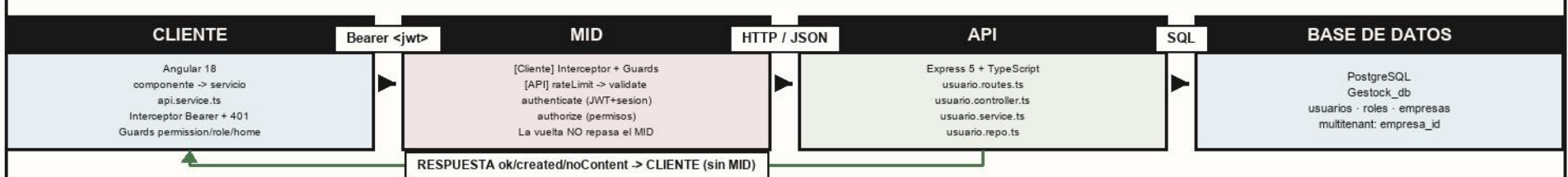
ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · USUARIO

GESTOCK · ARQUITECTURA Y FLUJOS

Usuarios / Usuarios · CRUD de usuarios: listar, detalle, crear, actualizar, activar/desactivar, cambiar rol, password, eliminar.



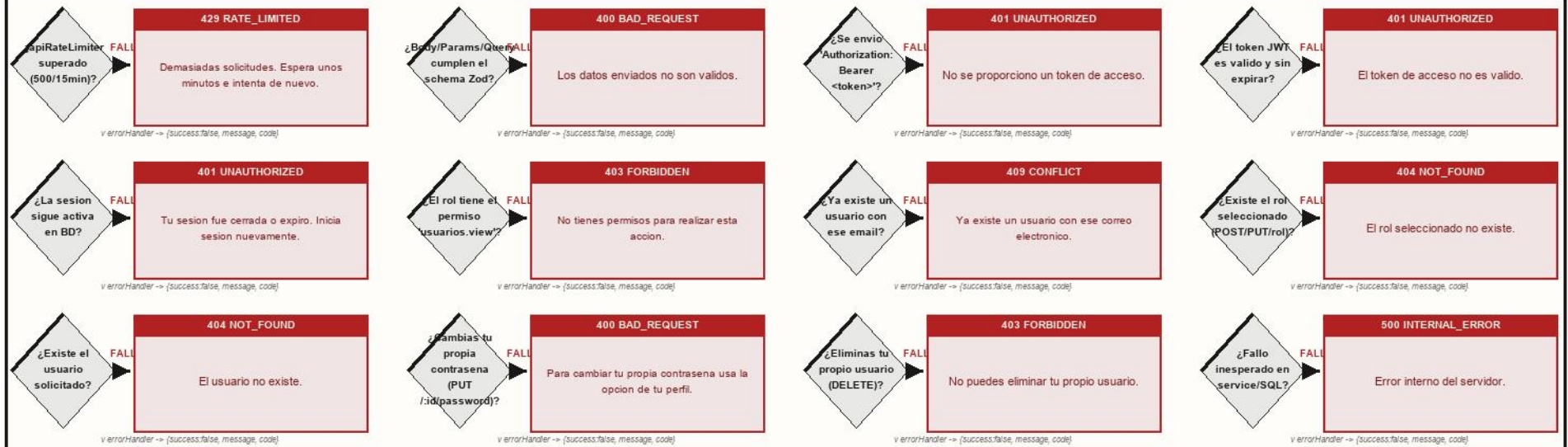
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/usuarios	authorizePermission('usuarios.view')
GET	/usuarios/{id}	authorizePermission('usuarios.view') + validateParams
POST	/usuarios	authorizePermission('usuarios.create') + validateBody
PUT	/usuarios/{id}	authorizePermission('usuarios.edit') + validateBody + validateParams
PATCH	/usuarios/{id}/estado	authorizePermission('usuarios.desactivar')
PATCH	/usuarios/{id}/rol	authorizePermission('usuarios.edit') + validateBody
PUT	/usuarios/{id}/password	authorizePermission('usuarios.password') + validateBody
DELETE	/usuarios/{id}	authorizePermission('usuarios.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL

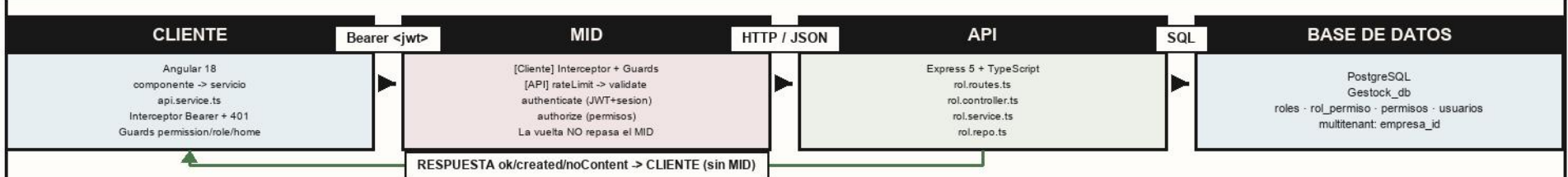


ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · ROL GESTOCK · ARQUITECTURA Y FLUJOS

Roles / Roles · CRUD de roles + asignacion de permisos (RBAC): listar, basico, detalle, crear, actualizar, estado, permisos.



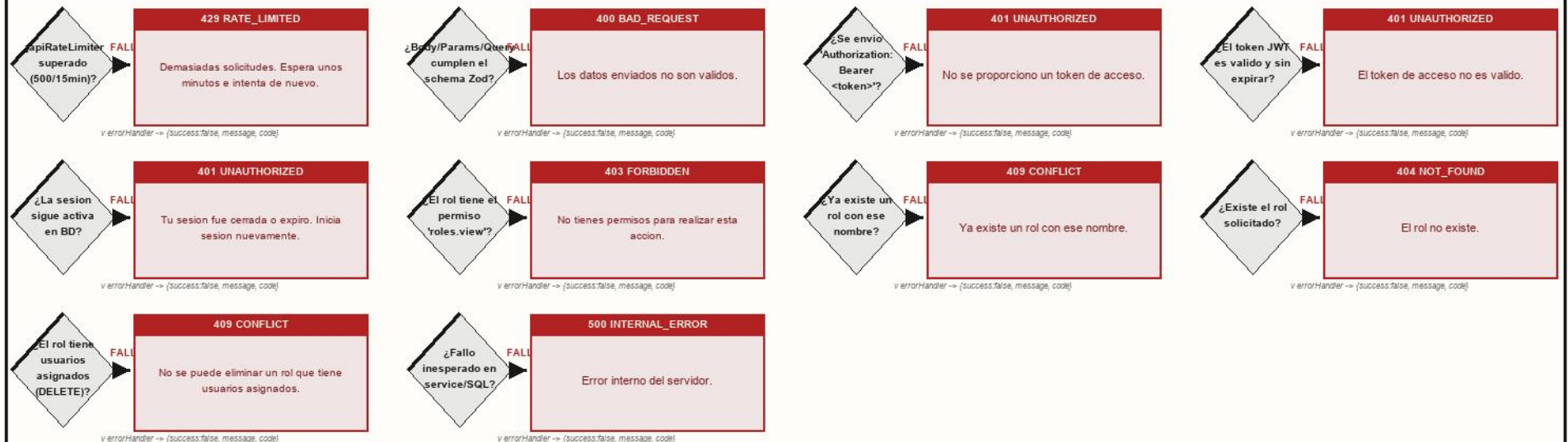
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/roles/basicos	authorizePermission('roles.view')
GET	/roles	authorizePermission('roles.view')
GET	/roles/id	authorizePermission('roles.view') + validateParams
POST	/roles	authorizePermission('roles.create') + validateBody
PUT	/roles/id	authorizePermission('roles.edit') + validateBody
PATCH	/roles/id/estado	authorizePermission('roles.edit') + validateBody
PUT	/roles/id/permisos	authorizePermission('roles.asignar_permisos') + validateBody
DELETE	/roles/id	authorizePermission('roles.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



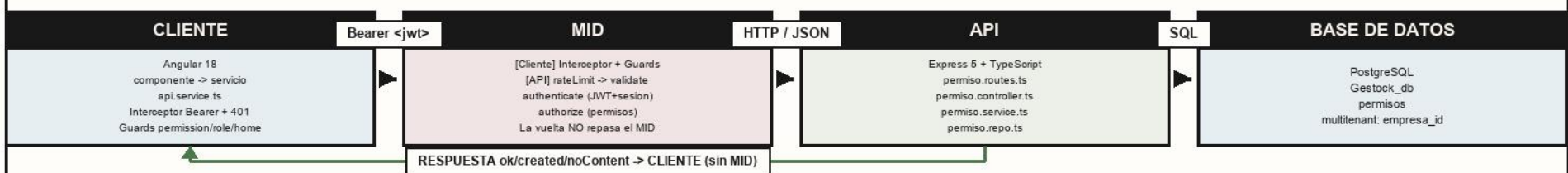
ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · PERMISO

GESTOCK · ARQUITECTURA Y FLUJOS

Permisos / Permisos · Catalogo de permisos: listar todos y listar por modulo.



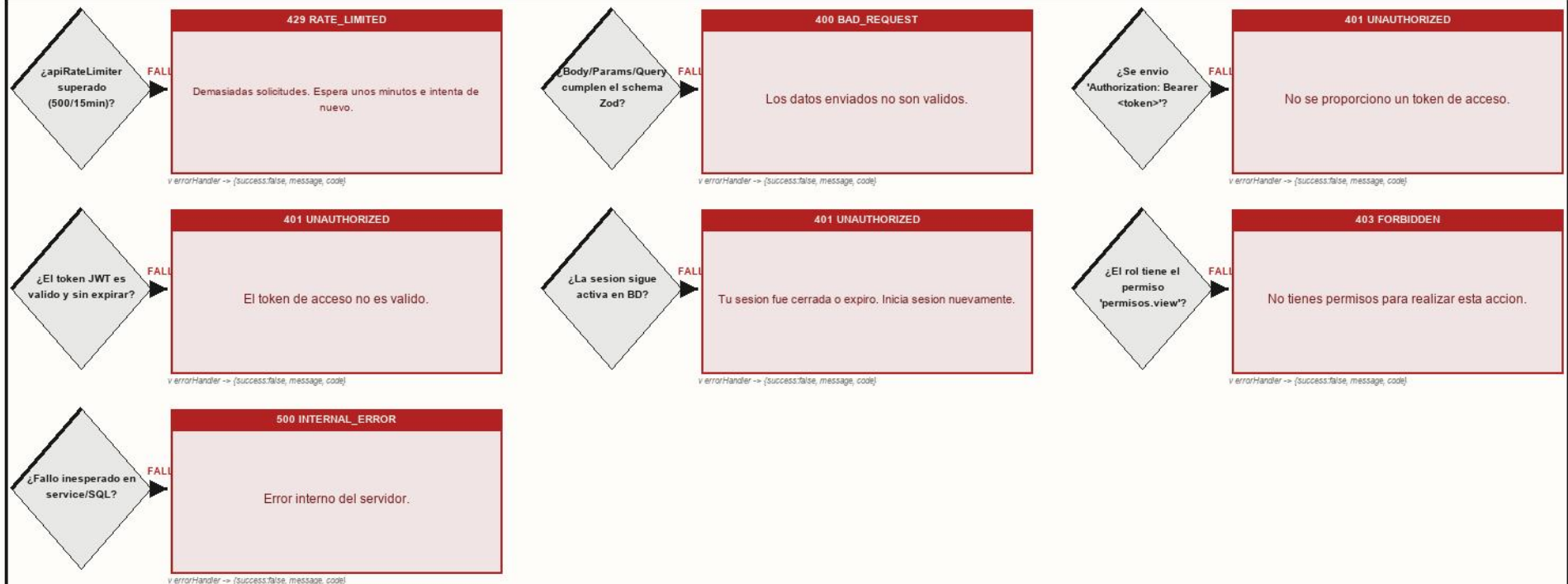
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/permisos	authorizePermission('permisos.view')
GET	/permisos/modulo/modulo	authorizePermission('permisos.view')

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL

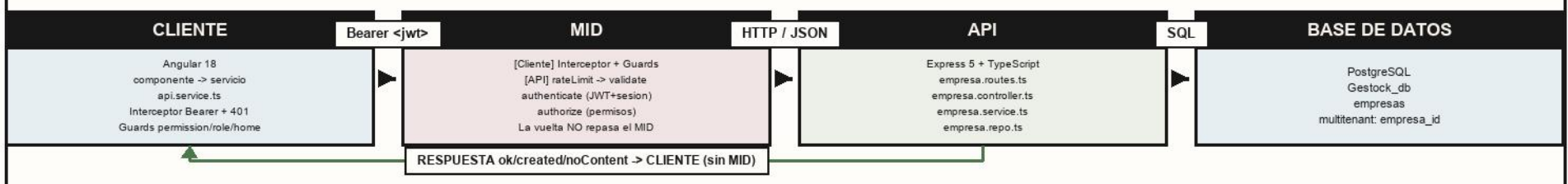


ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · EMPRESA GESTOCK · ARQUITECTURA Y FLUJOS

Empresas / Empresas · CRUD de empresas + seleccionar empresa activa (multiempresa).



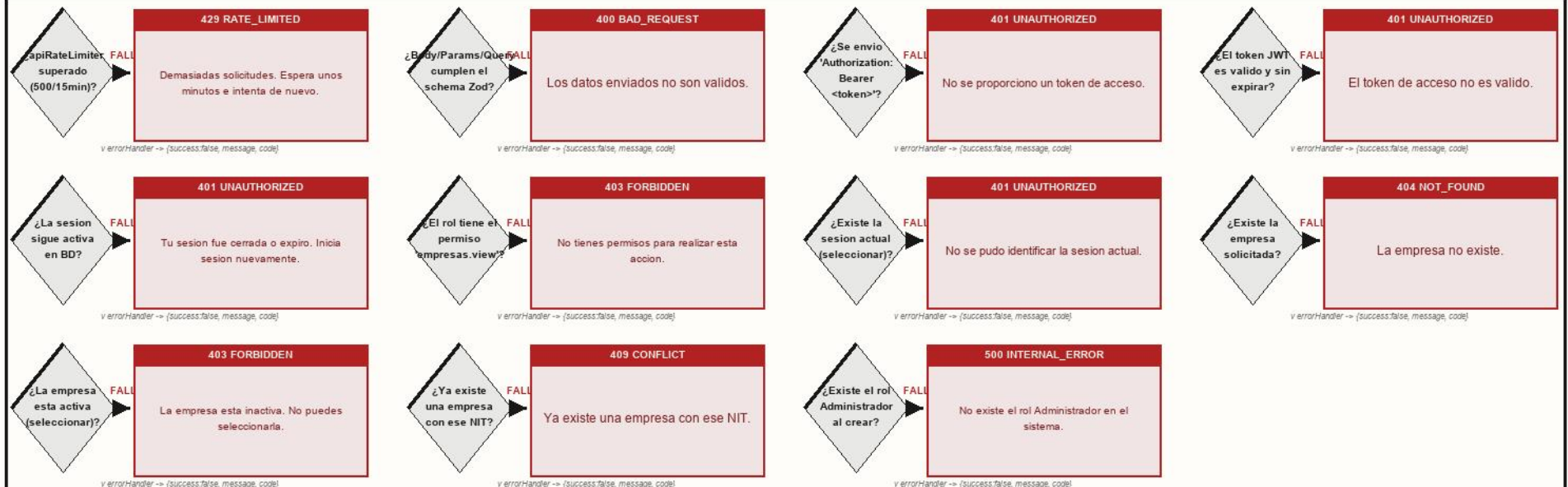
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
POST	/empresas/seleccionar	authorizePermission('empresas.view') + validateBody
GET	/empresas	authorizePermission('empresas.view')
GET	/empresas/id	authorizePermission('empresas.view') + validateParams
POST	/empresas	authorizePermission('empresas.create') + validateBody
PUT	/empresas/id	authorizePermission('empresas.edit') + validateBody
DELETE	/empresas/id	authorizePermission('empresas.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL

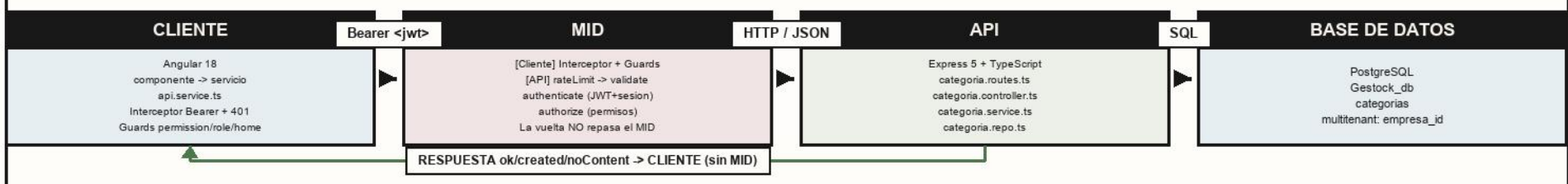


ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · GESTOCK · ARQUITECTURA Y FLUJOS

Categorias / Categorías · CRUD de categorías de productos.



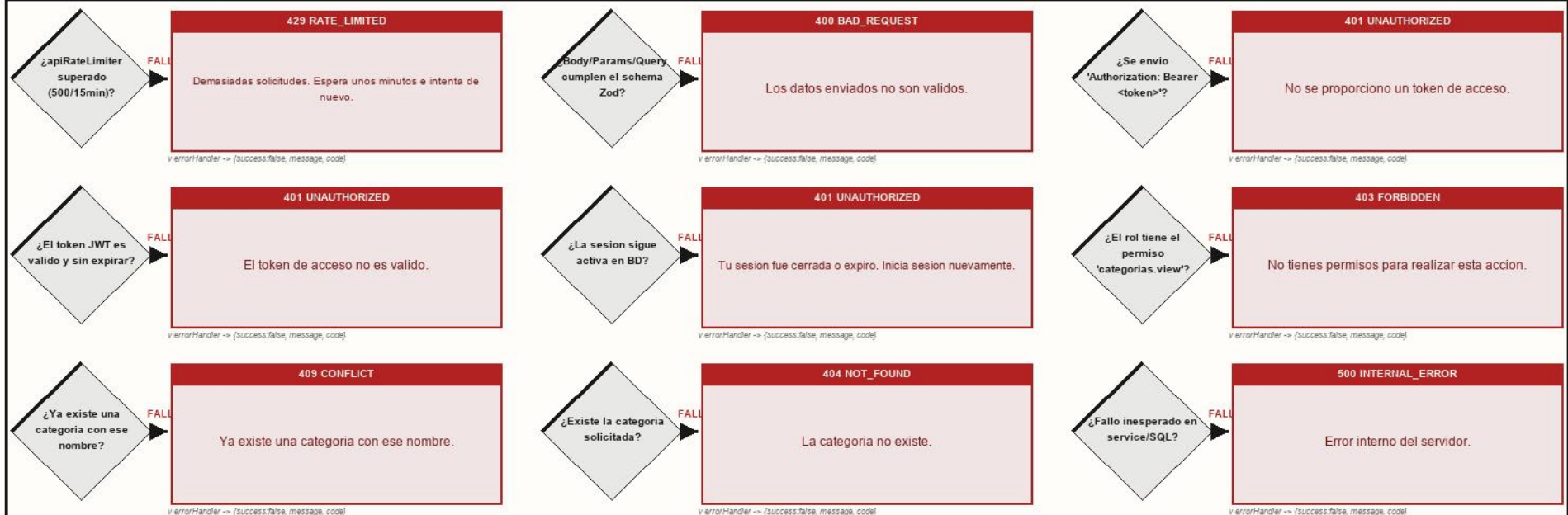
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/categorias	authorizePermission('categorias.view')
GET	/categorias/id	authorizePermission('categorias.view') + validateParams
POST	/categorias	authorizePermission('categorias.create') + validateBody
PUT	/categorias/id	authorizePermission('categorias.edit') + validateBody
DELETE	/categorias/id	authorizePermission('categorias.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL

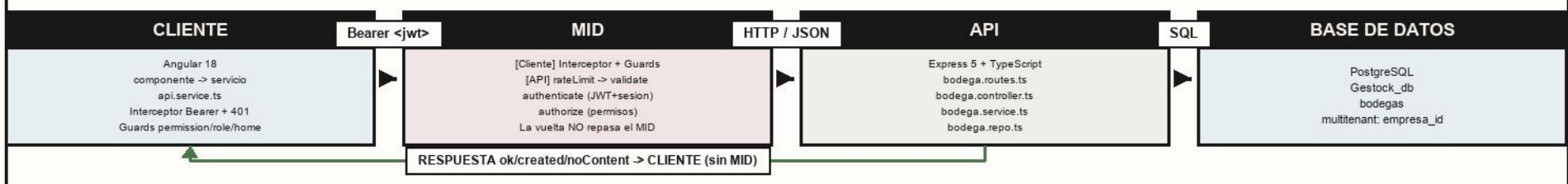


ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · BODEGA GESTOCK · ARQUITECTURA Y FLUJOS

Bodegas / Bodegas · CRUD de bodegas (almacenes).



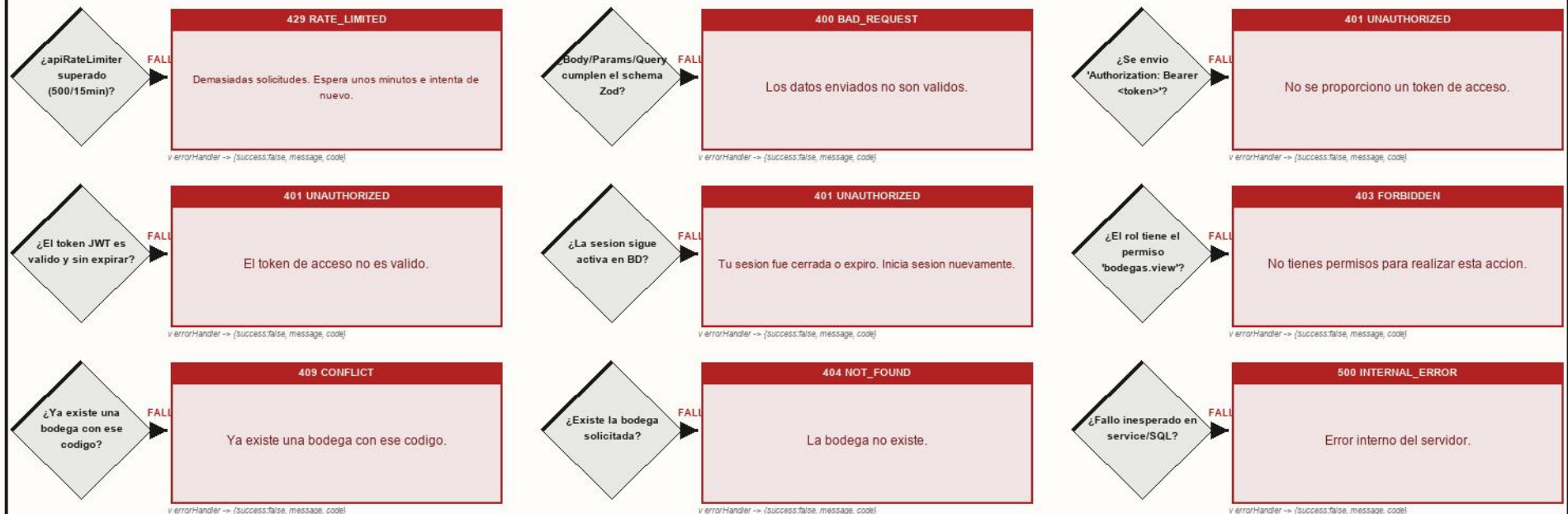
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/bodegas	authorizePermission('bodegas.view')
GET	/bodegas/id	authorizePermission('bodegas.view') + validateParams
POST	/bodegas	authorizePermission('bodegas.create') + validateBody
PUT	/bodegas/id	authorizePermission('bodegas.edit') + validateBody
DELETE	/bodegas/id	authorizePermission('bodegas.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



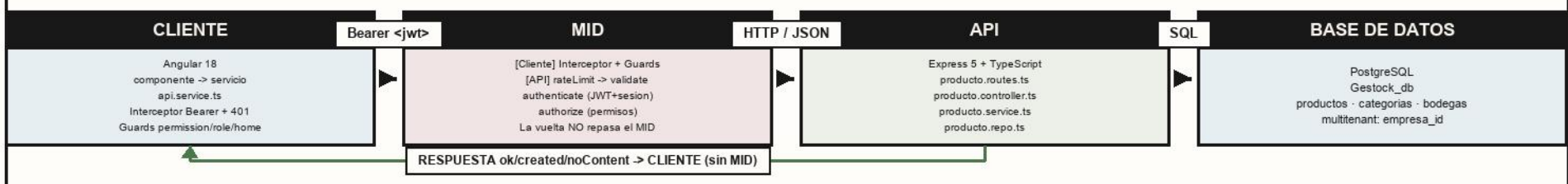
ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · PRODUCTO

GESTOCK · ARQUITECTURA Y FLUJOS

Productos / Productos · CRUD de productos + listado stock-bajo + detalle + actualizar precio/cantidad + eliminar.



RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/productos	authorizePermission('productos.view')
GET	/productos/stock-bajo	authorizePermission('inventario.view')
GET	/productos/:id	authorizePermission('productos.view') + validateParams
POST	/productos	authorizePermission('productos.create') + validateBody
PUT	/productos/:id	authorizePermission('productos.edit') + validateBody
PATCH	/productos/:id/estado	authorizePermission('productos.edit') + validateBody
DELETE	/productos/:id	authorizePermission('productos.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



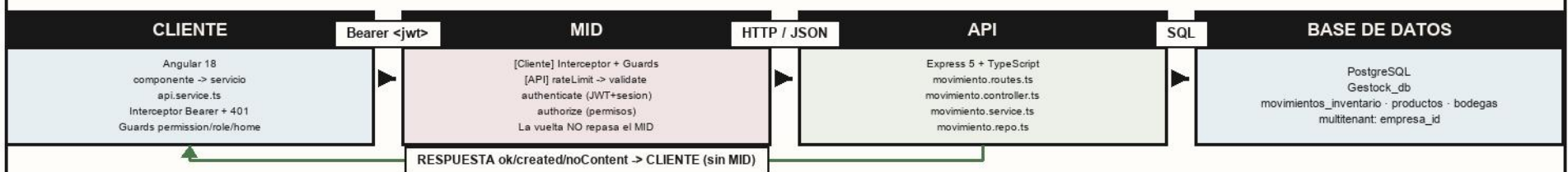
ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · MOVIMIENTO

GESTOCK · ARQUITECTURA Y FLUJOS

Movimientos / Movimientos de Inventario · Movimientos de inventario: listar con filtros, detalle, crear (entrada/salida/transferencia) y eliminar.



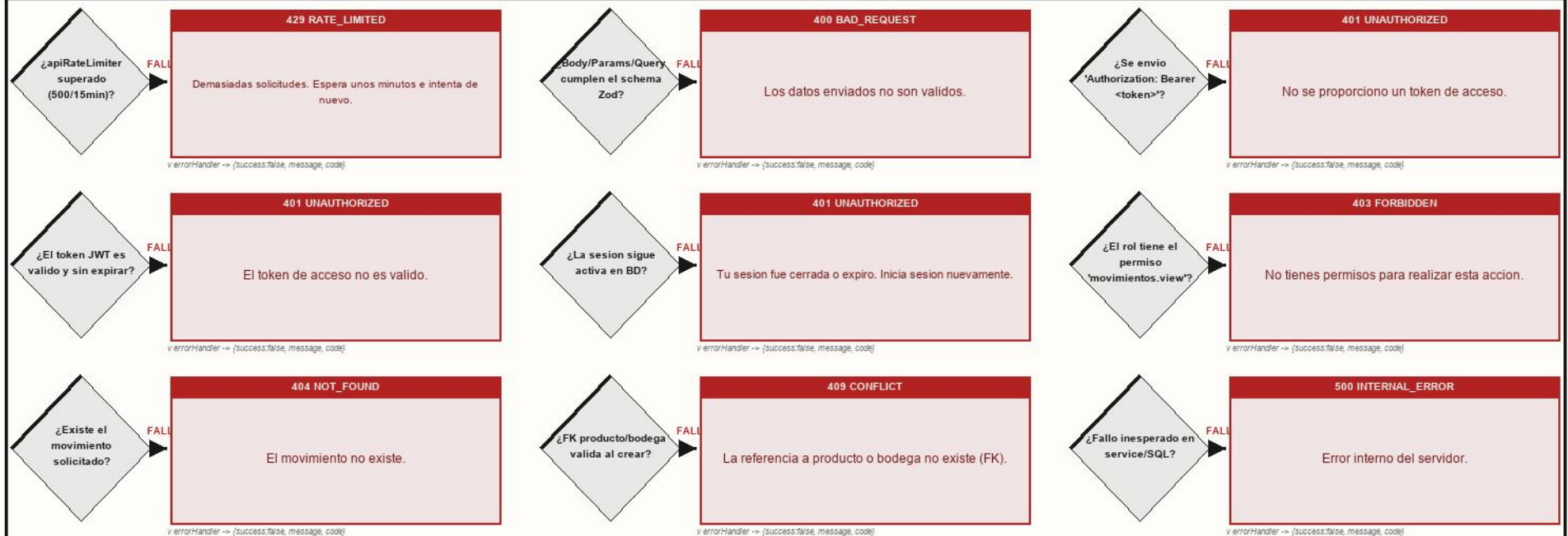
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/movimientos	authorizePermission('movimientos.view') + validateQuery
GET	/movimientos/:id	authorizePermission('movimientos.view') + validateParams
POST	/movimientos	authorizePermission('recepcion.create') + validateBody
DELETE	/movimientos/:id	authorizePermission('recepcion.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



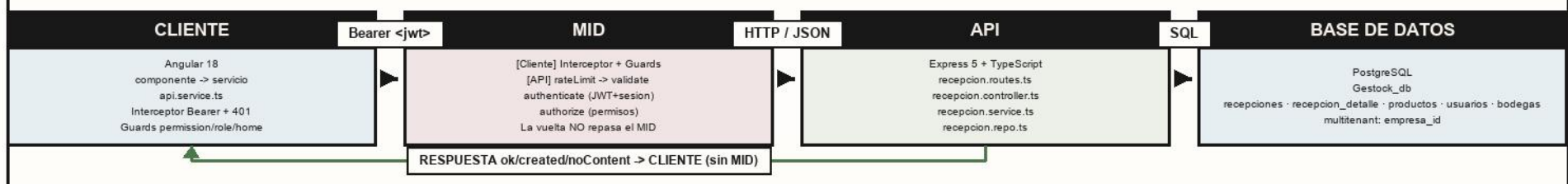
ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · RECEPCION

GESTOCK · ARQUITECTURA Y FLUJOS

Recepciones / Recepcion de Mercancia · Recepcion de mercancia con detalle de productos: listar, detalle, crear, actualizar, eliminar.



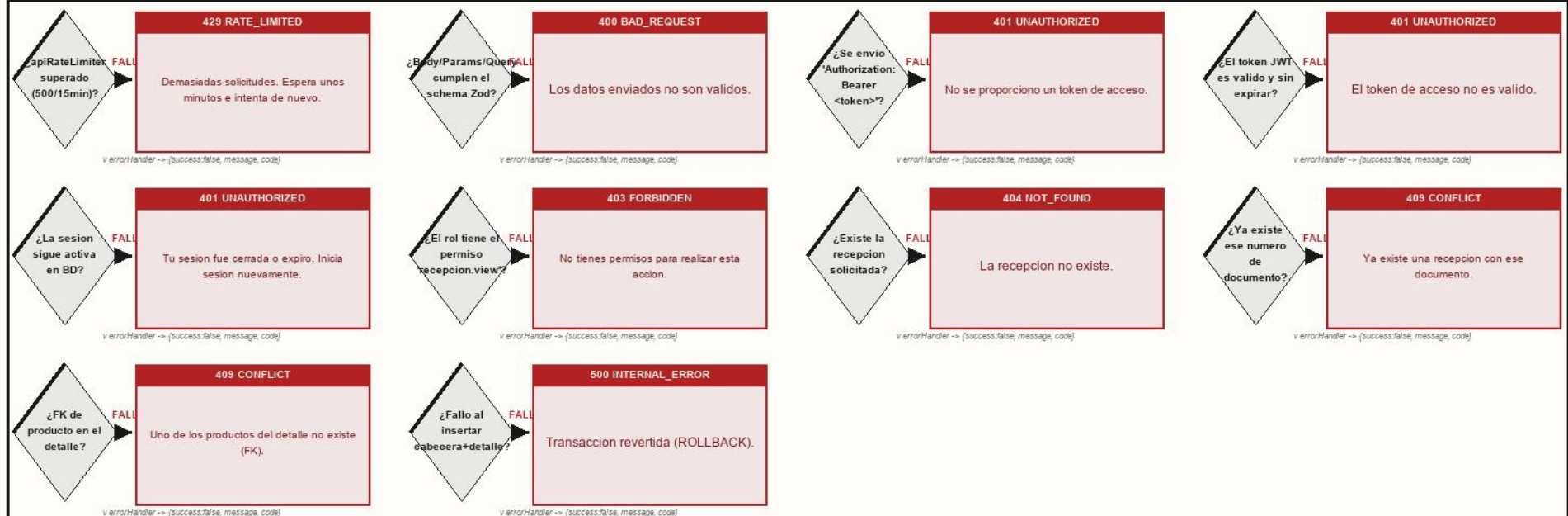
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/recepciones	authorizeAnyPermission('recepcion.view','historial.view')
GET	/recepciones/{id}	authorizeAnyPermission('recepcion.view','historial.view') + validateParams
POST	/recepciones	authorizePermission('recepcion.create') + validateBody
PUT	/recepciones/{id}	authorizePermission('recepcion.edit') + validateBody
DELETE	/recepciones/{id}	authorizePermission('recepcion.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



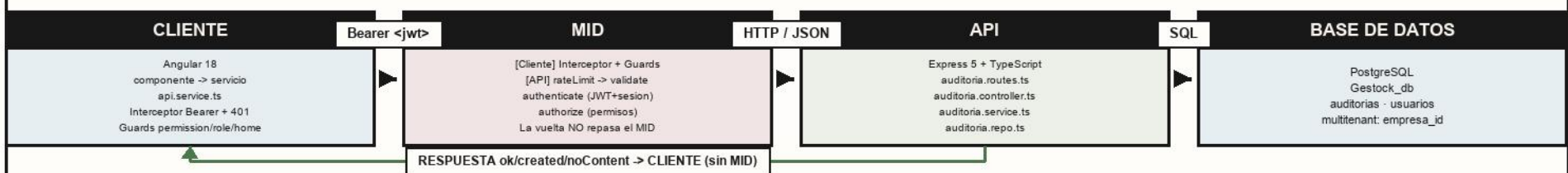
ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · AUDITORIA

GESTOCK · ARQUITECTURA Y FLUJOS

Auditorias / Auditoria · Registro de auditoria: listar y ver detalle por id.



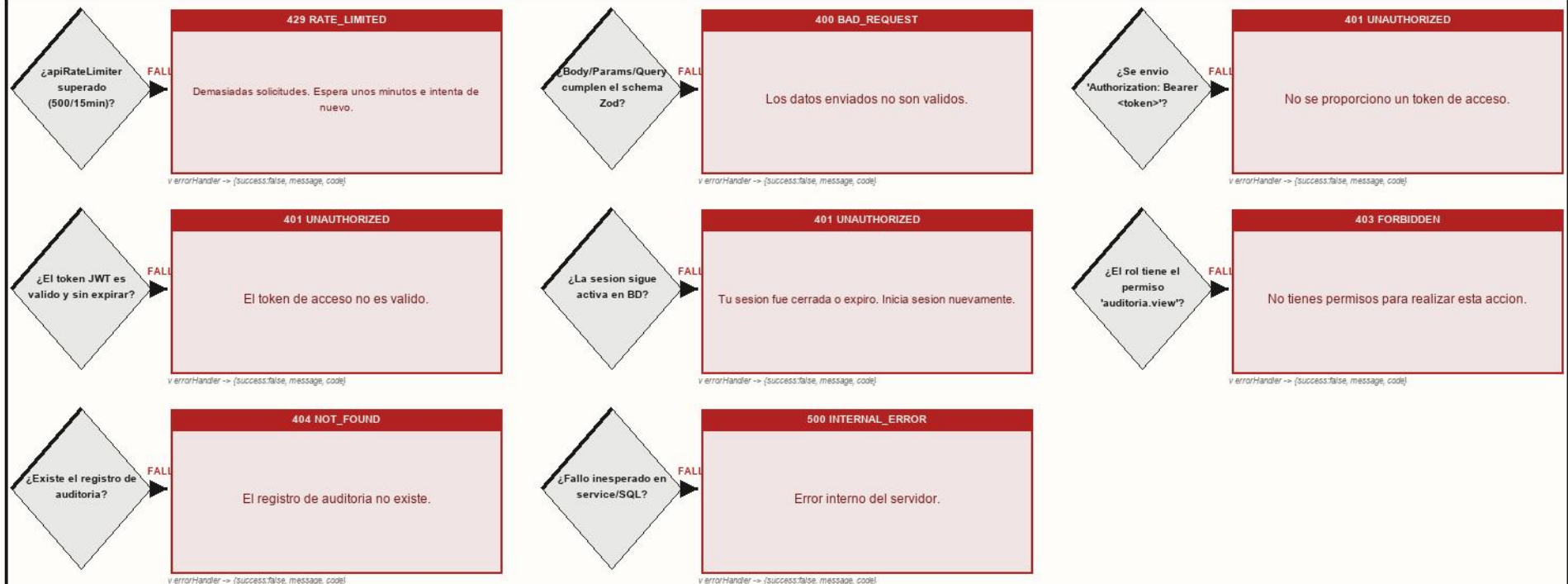
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/auditorias	authorizePermission('auditoria.view')
GET	/auditorias/id	authorizePermission('auditoria.view') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



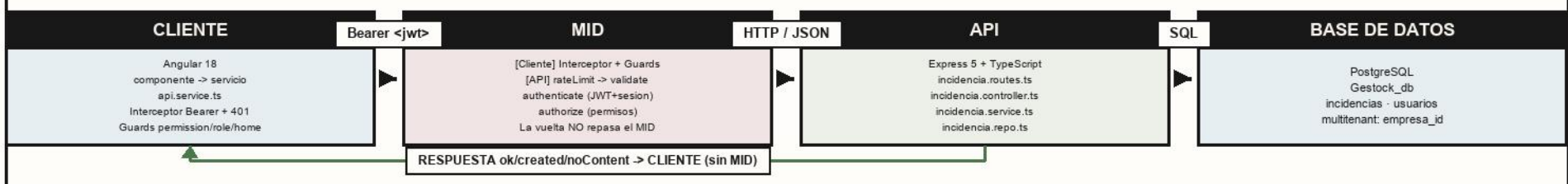
ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · INCIDENCIA

GESTOCK · ARQUITECTURA Y FLUJOS

Incidentes / Incidentes · Reporte de incidencias: listar, detalle, crear, actualizar y eliminar.



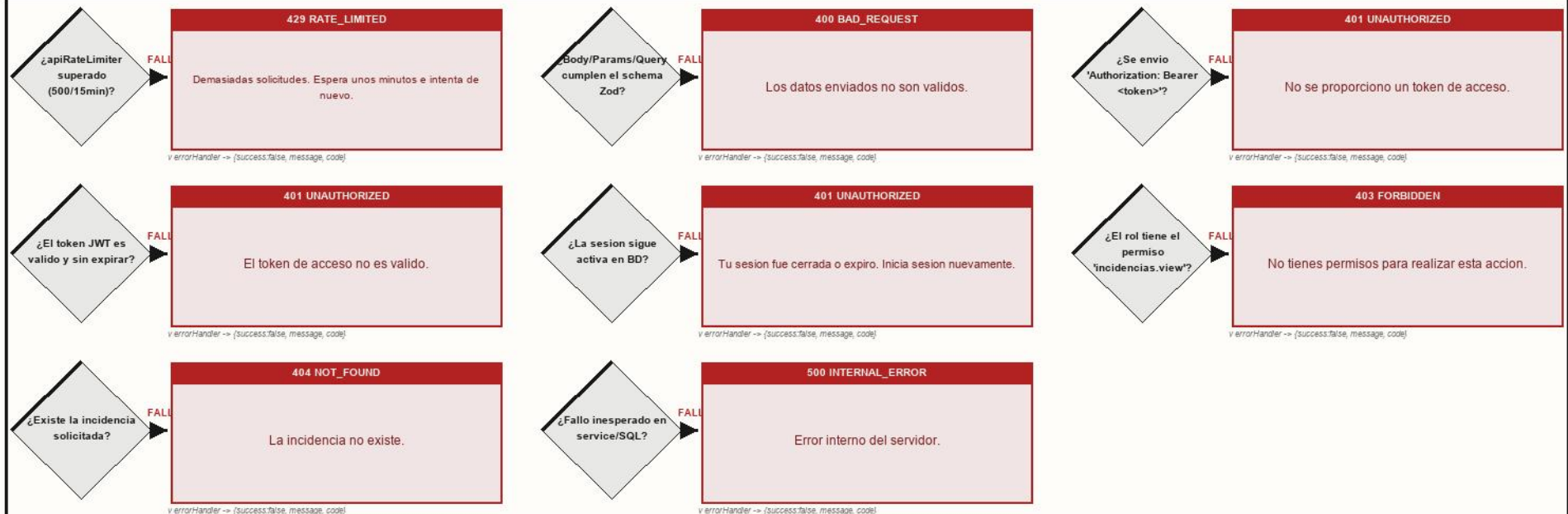
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/incidencias	authorizePermission('incidencias.view')
GET	/incidencias/id	authorizePermission('incidencias.view') + validateParams
POST	/incidencias	authorizePermission('incidencias.create') + validateBody
PUT	/incidencias/id	authorizePermission('incidencias.edit') + validateBody
DELETE	/incidencias/id	authorizePermission('incidencias.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL

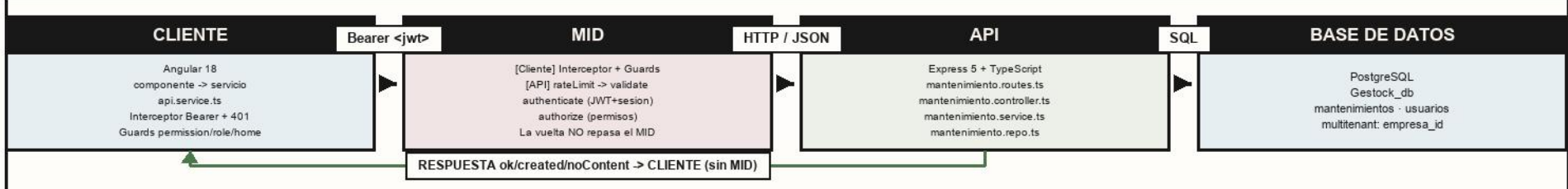


ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · MANTENIMIENTO

Mantenimientos / Mantenimiento · Mantenimientos de equipos: listar, detalle, crear, actualizar y eliminar.



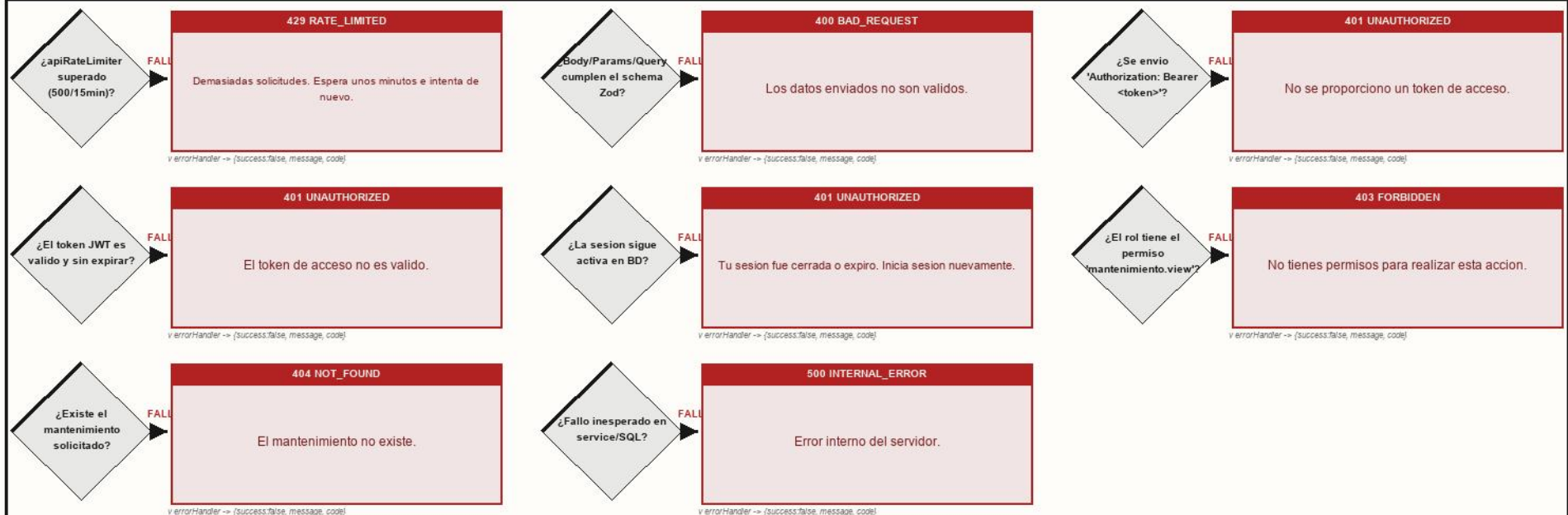
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/mantenimientos	authorizePermission('mantenimiento.view')
GET	/mantenimientos/:id	authorizePermission('mantenimiento.view') + validateParams
POST	/mantenimientos	authorizePermission('mantenimiento.create') + validateBody
PUT	/mantenimientos/:id	authorizePermission('mantenimiento.edit') + validateBody
DELETE	/mantenimientos/:id	authorizePermission('mantenimiento.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



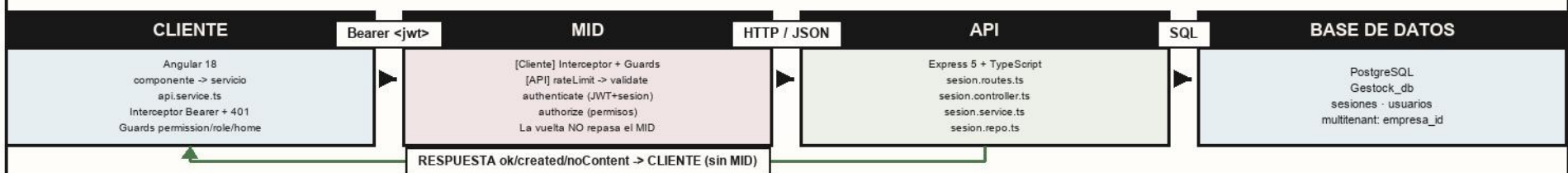
ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · SESION

GESTOCK · ARQUITECTURA Y FLUJOS

Sesiones / Sesiones · Sesiones activas: listar, cerrar todas excepto la actual y eliminar una sesion.



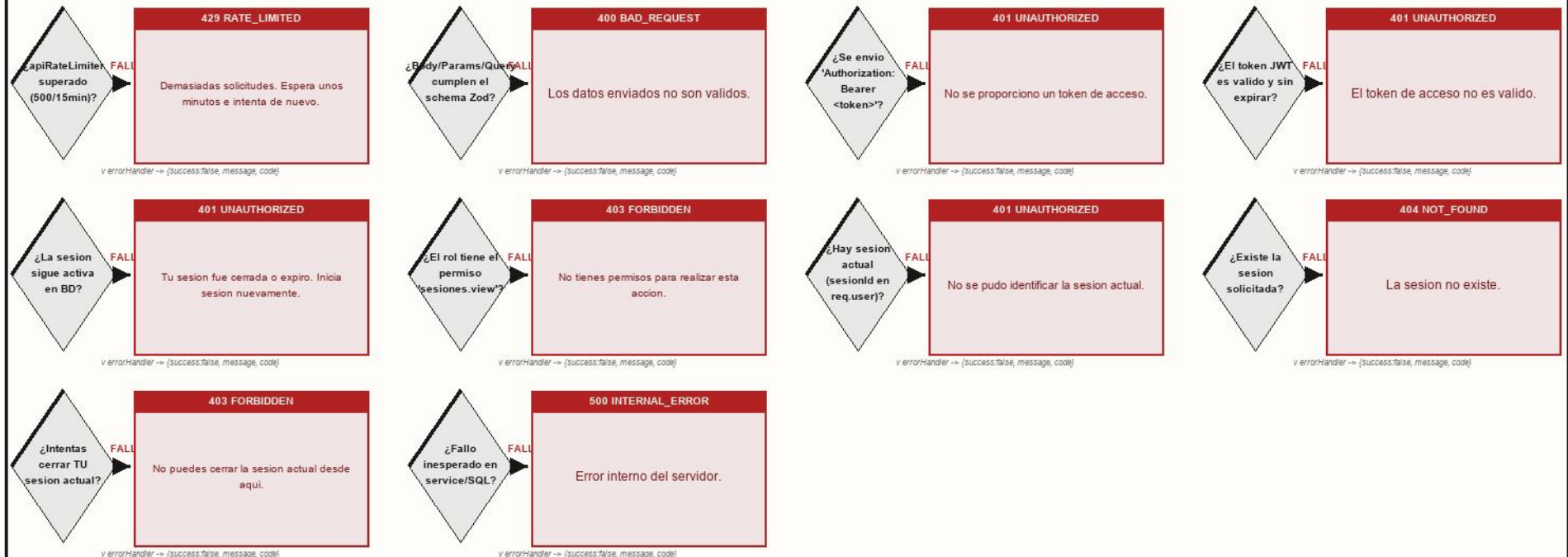
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/sesiones	authorizePermission('sesiones.view')
POST	/sesiones/cerrar-otras	authorizePermission('sesiones.delete')
DELETE	/sesiones/:id	authorizePermission('sesiones.delete') + validateParams

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



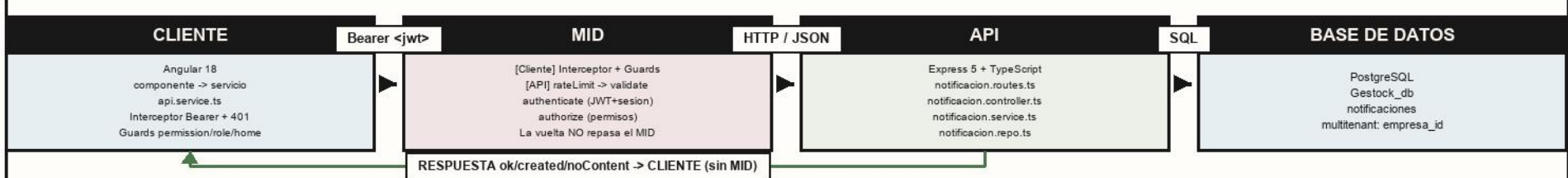
ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · NOTIFICACION

GESTOCK · ARQUITECTURA Y FLUJOS

Notificaciones / Notificaciones · Notificaciones: listar, marcar una como leida y marcar todas como leidas.



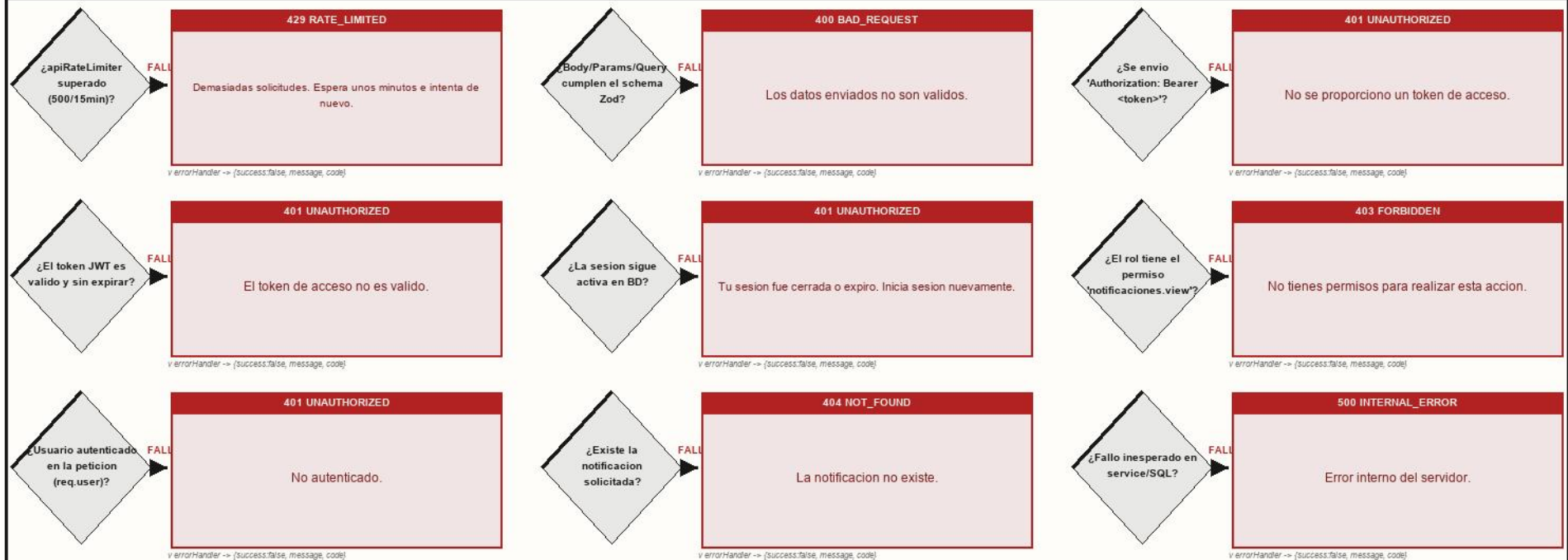
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/notificaciones	authorizePermission('notificaciones.view')
PATCH	/notificaciones/id/leida	authorizePermission('notificaciones.view') + validateBody
POST	/notificaciones/leidas	authorizePermission('notificaciones.view')

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL

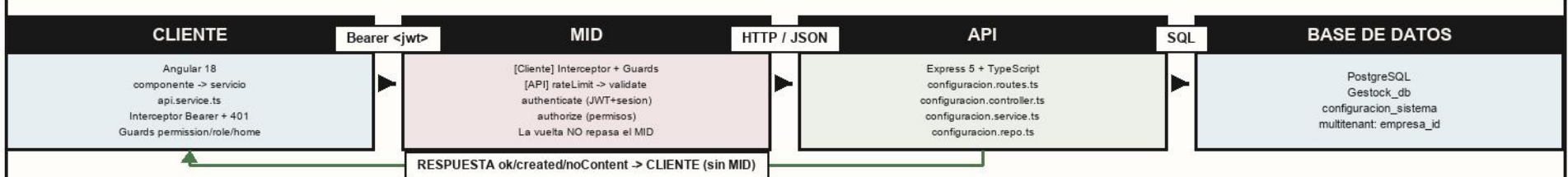


ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · CONFIGURACION

Configuracion / Configuracion del Sistema · Configuracion de la sesion: listar y actualizar valores.



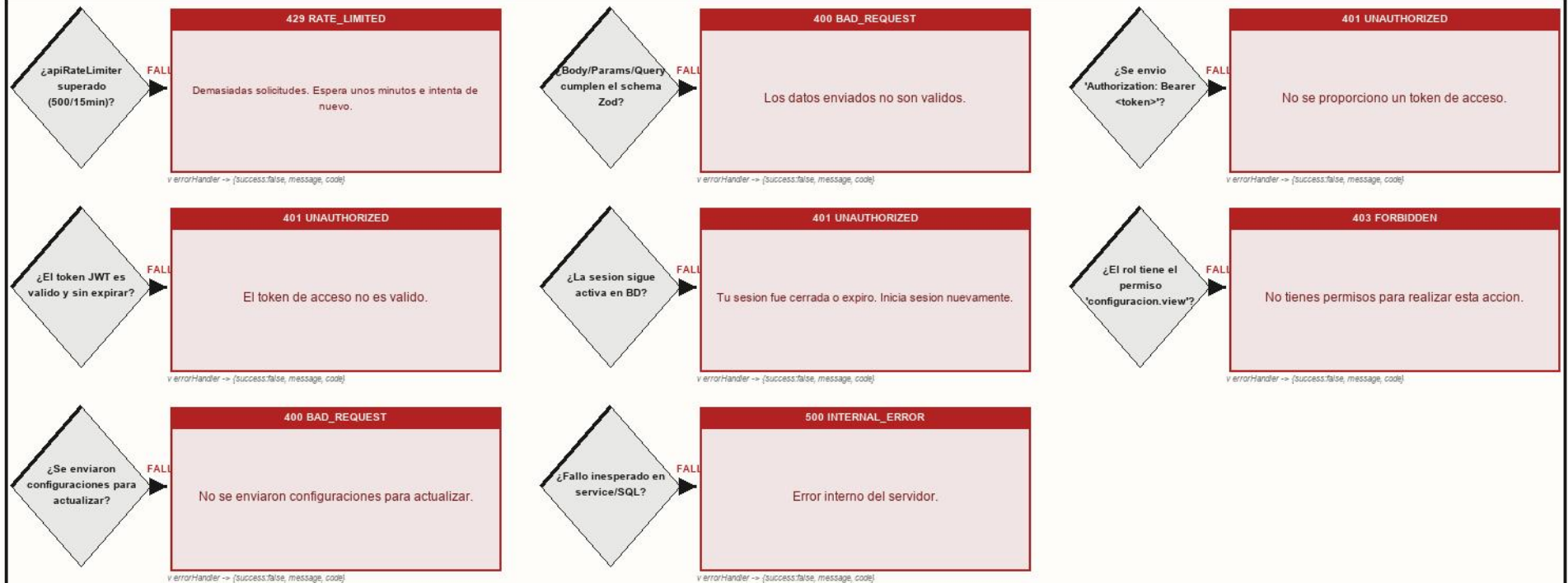
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/configuracion	authorizePermission('configuracion.view')
PUT	/configuracion	authorizePermission('configuracion.edit') + validateBody

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL

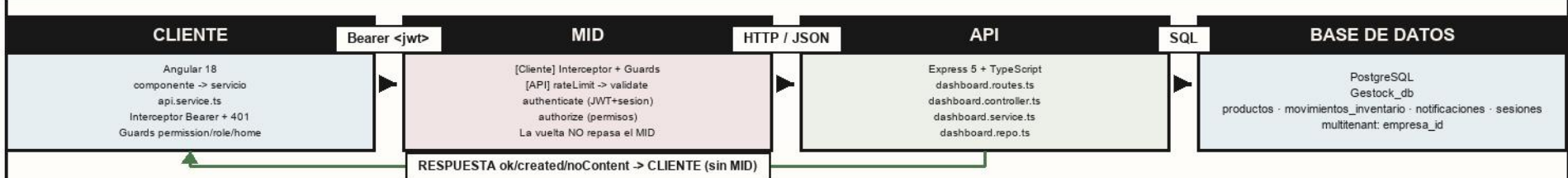


ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · DASHBOARD · GESTOCK · ARQUITECTURA Y FLUJOS

Dashboard / Panel Principal · Indicadores del dashboard: totales y resumen (productos, stock bajo, inventario, graficas).



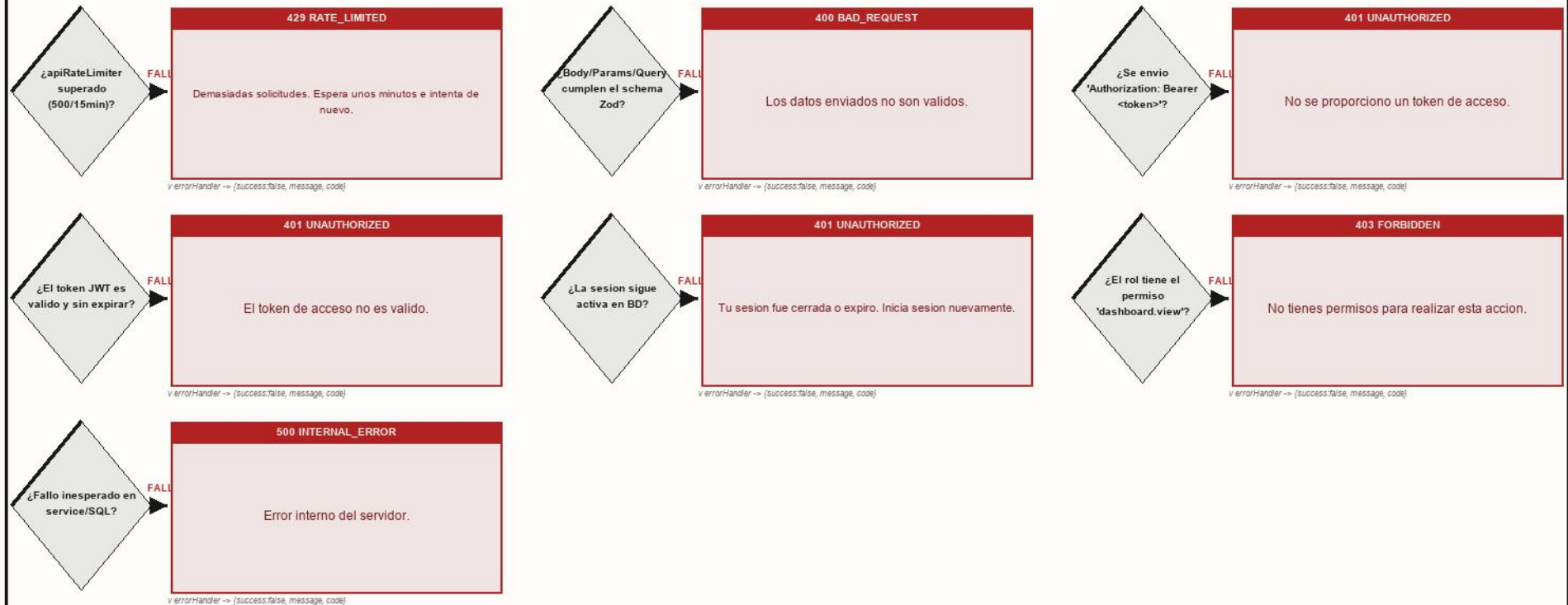
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/dashboard/resumen	authorizePermission('dashboard.view')

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



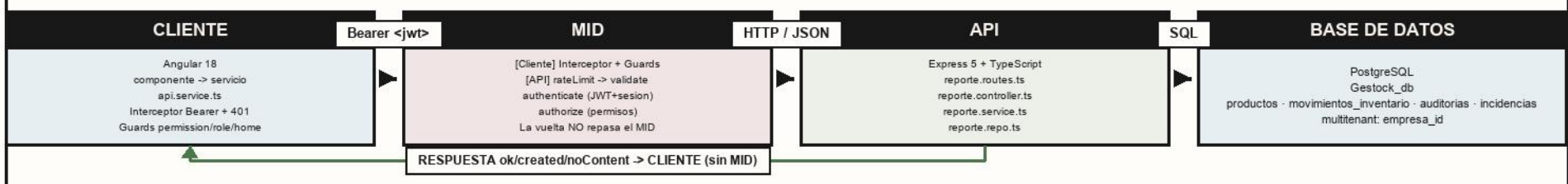
ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CLIENTE -> MID -> API -> BASE DE DATOS · REPORTE

GESTOCK · ARQUITECTURA Y FLUJOS

Reportes / Reportes · Reportes: inventario, movimientos, stock, auditorias e incidencias.



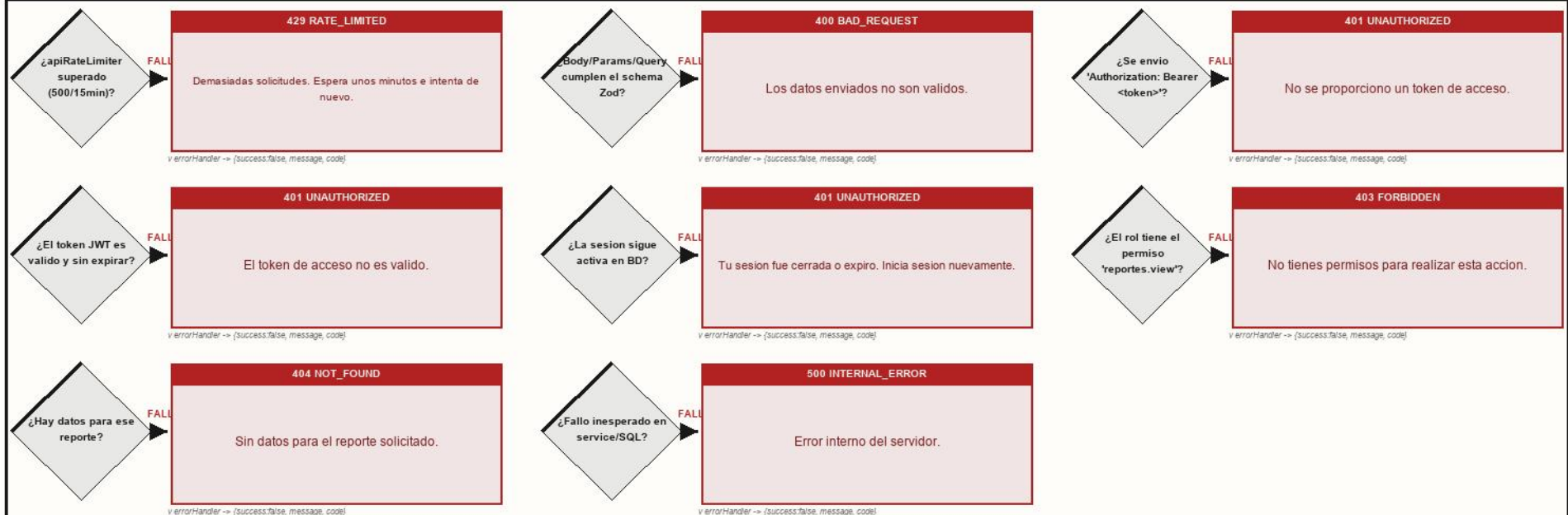
RUTAS, PERMISOS Y MIDDLEWARE (reales)

Metodo	Ruta	Permiso / Middleware
GET	/reportes/inventario	authorizePermission('reportes.view')
GET	/reportes/movimientos	authorizePermission('reportes.view')
GET	/reportes/stock	authorizePermission('reportes.view')
GET	/reportes/auditorias	authorizePermission('reportes.view')
GET	/reportes/incidencias	authorizePermission('reportes.view')

CAPA DE DATOS: ROUTER -> CONTROLLER -> SERVICE/REPO -> POSTGRESQL



ERRORES: UNA RAMA INDEPENDIENTE POR CADA FALLA (diamante -> falla -> errorHandler)



CICLO COMPLETO DE UNA PETICION: IDA CON MID, VUELTA SIN MID

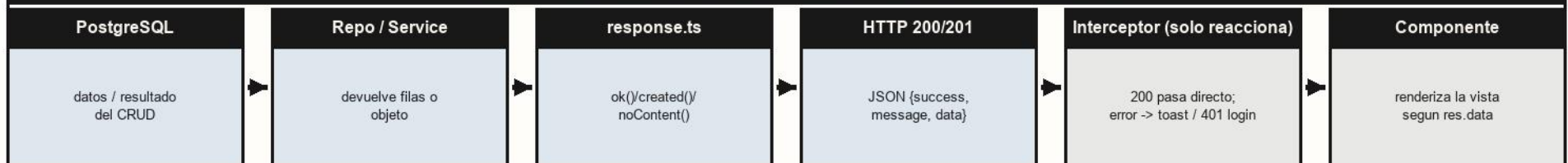
El MID valida la petición que ENTRA; la respuesta SALE directo del API al cliente

1) IDA (SOLICITUD) - EL MID INTEGRA AMBOS LADOS



El MID NO revalida nada en la vuelta: la respuesta no vuelve a pasar por authenticate/authorize/validate.

2) VUELTA (RESPUESTA) - CRUD -> CLIENTE SIN VOLVER POR EL MID



RETORNO API -> CLIENTE SIN MID (CRUD DIRECTO) GESTOCK · ARQUITECTURA Y FLUJOS

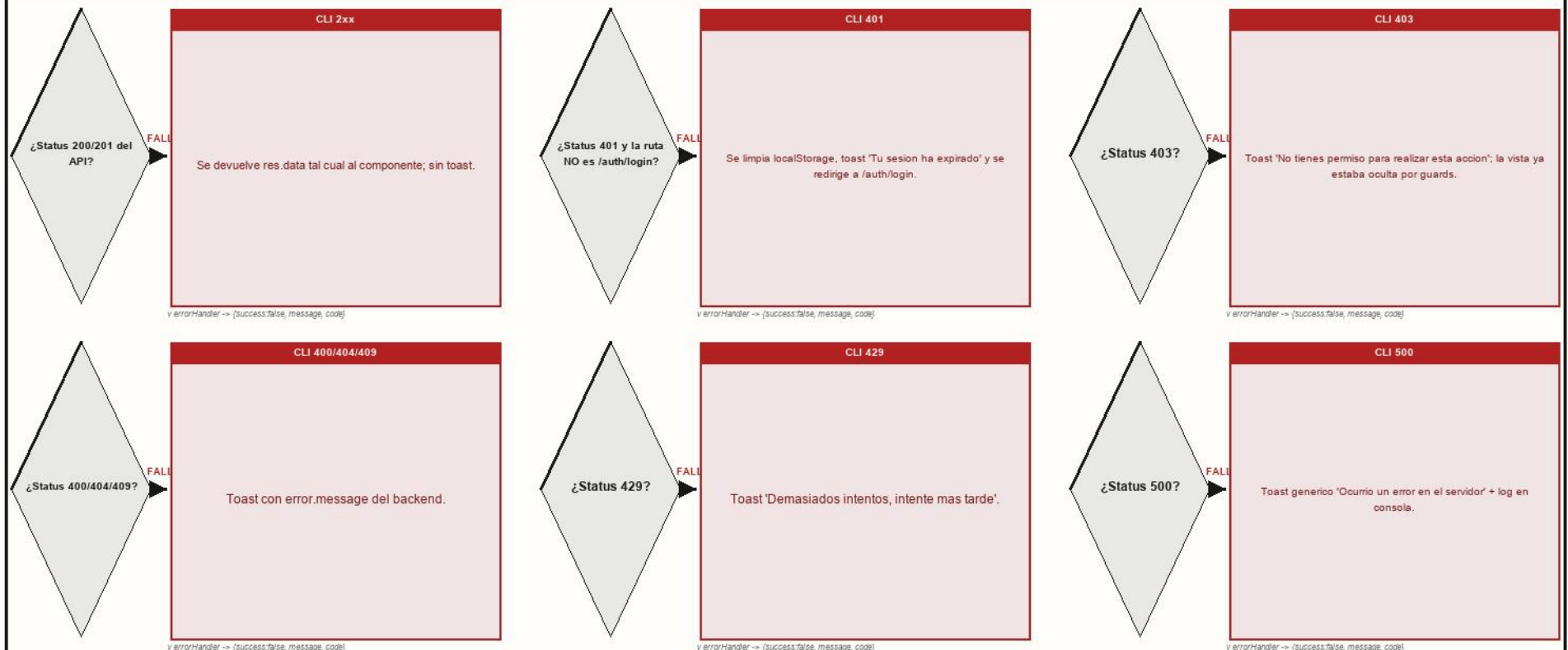
Solo los errores y el 401 reaccionan en el interceptor

RESPUESTAS DE EXITO (2xx) - LLEGAN TAL CUAL

ok()	created()	noContent()
200 {success:true, message, data}	201 {success:true, message, data}	200 {success:true, message, data:null}

Estas respuestas NO pasan por ninguna validacion del MID: llegan tal cual al componente.

RESPUESTAS DE ERROR (4xx/5xx) - CADA CODIGO SU PROPIA RAMA EN EL INTERCEPTOR



ENDPOINTS PUBLICOS: FLUJOS QUE NO REQUIEREN MID

GESTOCK · ARQUITECTURA Y FLUJOS

Login, recuperacion y documentacion acceden sin token (sin authenticate/authorize)

RUTAS QUE RESPONDEN AL CLIENTE SIN PASAR POR AUTHENTICATE/AUTHORIZE

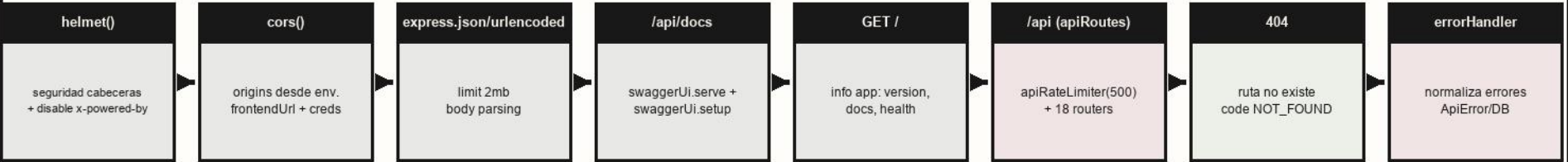
Ruta	Validacion / comportamiento
GET /	info de la app: version, docs, health
GET /api/docs	Swagger UI (swaggerUi.serve + setup)
GET /api/health	estado del servicio
POST /api/auth/login	validateBody(loginSchema) + loginEmailLimiter(5) + loginIpLimiter(60)
POST /api/auth/recuperar	recuperacionLimiter + validateBody(solicitarRecuperacionSchema)
POST /api/auth/recuperar/validar	validateBody(validarCodigoRecuperacionSchema)
POST /api/auth/recuperar/restablecer	validateBody(restablecerPasswordSchema)

FLUJO LOGIN SIN MID (solo validacion de forma y limites de intentos)



CADENA DE MIDDLEWARE DE app.ts (PUNTO DE ENTRADA UNICO)

Como se construye la aplicacion Express en backend/src/app.ts



DETALLE DE app.use('/api', apiRateLimiter, apiRoutes)

Concepto	Detalle
apiRoutes (routes/index.ts)	app.use('/api', apiRateLimiter, apiRoutes) · apiRoutes monta un router por controlador con prefijo /api
Rate limit global	apiRateLimiter: 500 peticiones / 15 min por IP -> 429 RATE_LIMITED
Routers reales	auth.routes.ts · usuario.routes.ts · rol.routes.ts · permiso.routes.ts · empresa.routes.ts · categoria.routes.ts · bodega.routes.ts · producto.routes.ts · movimiento.routes.ts · recepcion.routes.ts · auditoria.routes.ts · incidencia.routes.ts · mantenimiento.routes.ts · sesion.routes.ts · notificacion.routes.ts · configuracion.routes.ts · dashboard.routes.ts · reporte.routes.ts

RUTAS ANGULAR PROTEGIDAS POR GUARDS (MID CLIENTE)

app.routes.ts: canActivate + data.permission / anyPermission

RUTAS DEL FRONTEND CON SU GUARD Y PERMISO

Ruta	Guard	Permiso requerido
/ (pagina)	sin guard	publica - home / landing
/auth/login	sin guard	publica - login
/auth/recuperar-contrasena	sin guard	publica - recuperacion
/auth/crear-usuario	permissionGuard	data.permission: usuarios.create
/auth/sesiones-activas	permissionGuard	data.permission: sesiones.view
/app/panel	permissionGuard	data.permission: dashboard.view
/app/empresas	permissionGuard	data.permission: empresas.view
/app/registrar-empresa	permissionGuard	data.permission: empresas.create
/app/gestion/inventario/lista-productos	permissionGuard	data.permission: productos.view
/app/gestion/inventario/registrar-productos	permissionGuard	data.permission: productos.create
/app/gestion/inventario/bodegas	permissionGuard	data.permission: bodegas.view
/app/gestion/inventario/categorias	permissionGuard	data.permission: categorias.view
/app/recepcion/recepcion-mercancias	permissionGuard	data.anyPermission: [recepcion.view, recepcion.create]
/app/recepcion/historial-logistico	permissionGuard	data.permission: historial.view
/app/gestion/auditorias	permissionGuard	data.permission: auditoria.view
/app/gestion/roles-yusuarios	permissionGuard	data.anyPermission: [usuarios.view, roles.view]
/app/reportes	permissionGuard	data.permission: reportes.view
/app/configuracion	permissionGuard	data.permission: configuracion.view
/app/programacion	permissionGuard	data.anyPermission: [mantenimiento.view, mantenimiento.create]
/app/incidencias	permissionGuard	data.anyPermission: [incidencias.view, incidencias.create]

MAPA: PAGINA FRONTEND -> ENDPOINTS QUE LLAMA (CON/SIN MID)

Cada pagina Angular consulta su(s) recurso(s) via ApiService (11 llamadas directas listadas)

Todas las llamadas pasan por apiInterceptor (Bearer + manejo de errores). Las rutas publicas (login, recuperacion) no tienen token: el interceptor no adjunta Authorization.

Pagina	Endpoints llamados	MID?
Login	POST /api/auth/login	publica (sin token)
Recuperacion de contraseña	POST /auth/recuperar · POST /auth/recuperar/validar · POST /auth/recuperar/restablecer	publica (sin token)
Crear usuario	GET /roles/basicos · POST /usuarios	MID (Bearer + permisos)
Sesiones activas	GET /sesiones · POST /sesiones/cerrar-otras · DELETE /sesiones/:id	MID
Panel	GET /dashboard/resumen · GET /empresas	MID
Empresas	GET /empresas · POST /empresas · POST /empresas/seleccionar	MID
Registrar empresa	POST /empresas · POST /empresas/seleccionar	MID
Inventario	GET/POST/PUT/PATCH/DELETE /productos · GET /productos/stock-bajo · GET /categorias · GET /bodegas	MID
Recepcion de mercancías	GET/POST /productos, /bodegas, /movimientos y /recepciones	MID
Historial logístico	GET /movimientos · GET /recepciones	MID
Auditorías	GET /auditorias	MID
Roles y usuarios	GET /usuarios · GET /roles · GET /roles/basicos · GET /permisos · PUT /roles/:id/permisos	MID
Configuración	GET /configuracion · PUT /configuracion · GET /sesiones	MID
Mantenimiento	GET/POST/PUT/DELETE /mantenimientos	MID
Incidencias	GET/POST/PUT/DELETE /incidencias	MID
Reportes	GET /reportes/inventario · /movimientos · /stock · /auditorias · /incidencias	MID
Header	GET /notificaciones · POST /notificaciones/leidas	MID

MANEJO DE ERRORES EN EL CLIENTE (frontend)

GESTOCK · ARQUITECTURA Y FLUJOS

Que hace Angular cuando la API devuelve cada codigo · cada codigo su propia rama

REACCION DEL INTERCEPTOR POR CADA CODIGO (api.interceptor.ts)

